Latent Semantic Analysis and Latent Semantic Indexing

CS 115B: Fundamentals of NLP

Prof. James Pustejovsky

Brandeis University

Spring 2026

Outline

1. Motivation
2. The Term-Document Matrix
3. Singular Value Decomposition
4. Truncated SVD and Dimensionality Reduction
5. LSI: Application to Information Retrieval
6. LSA for Semantic Similarity
7. Limitations and Extensions
8. Summary

The Problem with Keyword Matching

Traditional Information Retrieval:

- Match query terms directly to document terms
- Relies on exact lexical overlap

Two fundamental problems:

1. **Synonymy**: Different words, same meaning
   - Query: "automobile" misses documents about "car"
2. **Polysemy**: Same word, different meanings
   - Query: "bank" retrieves both financial and river documents

Can we discover latent semantic structure that transcends surface forms?

The Distributional Hypothesis

Harris (1954)

"Words that occur in similar contexts tend to have similar meanings."

Key insight: Word meaning can be inferred from patterns of co-occurrence.

From distribution to semantics:

- "dog" and "cat" appear in similar contexts (pet, feed, veterinarian, ...)
- "bank" (financial) co-occurs with: money, account, deposit, loan
- "bank" (river) co-occurs with: river, water, shore, flood

LSA operationalizes this hypothesis using linear algebra.

Building the Term-Document Matrix

Starting point: A corpus of m documents containing n unique terms.

Term-Document Matrix $\mathbf{A} \in \mathbb{R}^{n \times m}$:

$$A_{ij} = \text{(weighted) frequency of term } i \text{ in document } j$$

	d_1	d_2	d_3	d_4	d_5
car	2	0	1	0	1
bank	0	3	0	2	0
drive	1	1	2	0	1
money	0	2	0	3	0
road	1	0	1	0	2

Each **row** = term vector (term's distribution across documents)
Each **column** = document vector (document's term profile)

Weighting Schemes: TF-IDF

Raw counts are problematic:

- Common words ("the", "is") dominate
- Document length effects

TF-IDF weighting:

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \text{idf}(t)$$

where:

$$\text{tf}(t, d) = \log(1 + f_{t,d}) \quad \text{(log-scaled term frequency)}$$

$$\text{idf}(t) = \log\left(\frac{N}{|\{d : t \in d\}|}\right) \quad \text{(inverse document frequency)}$$

Effect: Upweights distinctive terms; downweights common terms.

SVD: The Mathematical Foundation

Singular Value Decomposition

Any matrix $\mathbf{A} \in \mathbb{R}^{n \times m}$ can be factored as:

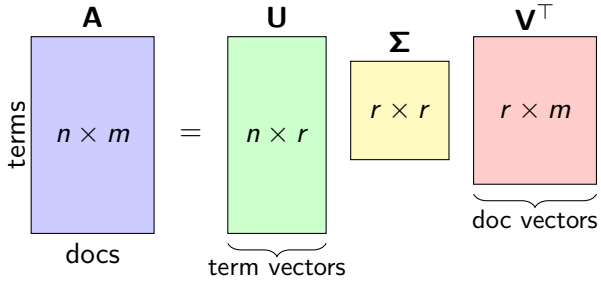
$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$$

$\mathbf{U} \in \mathbb{R}^{n \times r}$	Left singular vectors (term space)
$\mathbf{\Sigma} \in \mathbb{R}^{r \times r}$	Diagonal matrix of singular values
$\mathbf{V} \in \mathbb{R}^{m \times r}$	Right singular vectors (document space)
$r = \text{rank}(\mathbf{A})$	Number of non-zero singular values

Properties:

- Columns of $\mathbf{U}$ are orthonormal: $\mathbf{U}^\top\mathbf{U} = \mathbf{I}$
- Columns of $\mathbf{V}$ are orthonormal: $\mathbf{V}^\top\mathbf{V} = \mathbf{I}$
- Singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0$

Visualizing SVD



Interpretation:

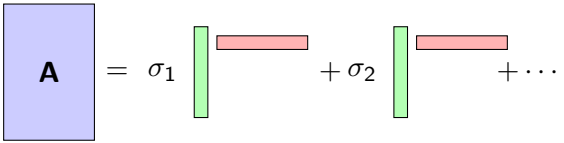
- **U**: Each row gives term i's coordinates in latent space
- **V**: Each row gives document j's coordinates in latent space
- **Σ**: Importance of each latent dimension

SVD as Sum of Rank-1 Matrices

The SVD can be written as a sum of outer products:

$$\mathbf{A} = \sum_{i=1}^{r} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^\top$$

Each term $\sigma_i \mathbf{u}_i \mathbf{v}_i^\top$ is a rank-1 matrix representing one "latent concept."

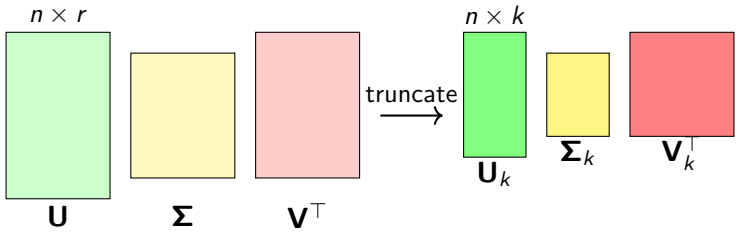


Singular values determine the "importance" of each latent dimension.

The Key Idea: Truncated SVD

LSA's core operation: Keep only the top k singular values/vectors.

$$\mathbf{A}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top = \sum_{i=1}^{k} \sigma_i \, \mathbf{u}_i \mathbf{v}_i^\top$$



Eckart-Young-Mirsky Theorem

$\mathbf{A}_k$ is the *best* rank-k approximation to $\mathbf{A}$ (minimizes Frobenius norm of error)

Why Does Truncation Help?

Noise reduction:

- Small singular values often correspond to noise
- Removing them smooths the representation

Latent semantic discovery:

- Forces terms with similar distributions to share dimensions
- "car" and "automobile" will have similar k-dimensional representations
- Polysemous words get "averaged" representations

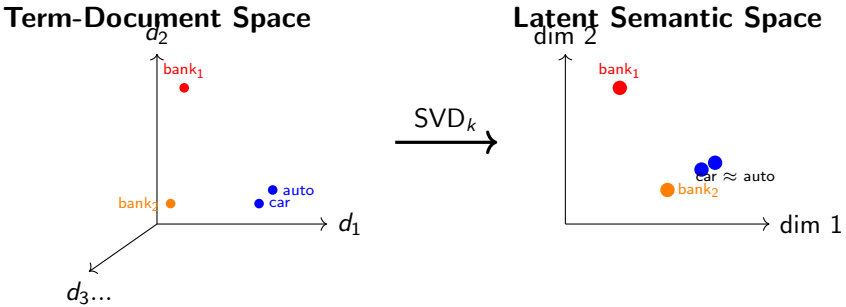
Practical benefits:

- Reduces storage: $O(nm) \to O((n+m)k)$
- Faster similarity computations
- Better generalization (less overfitting to specific term choices)

Typical values: $k \in [100, 500]$ for large corpora.

Geometric Interpretation

Original space: Terms and documents in high-dimensional sparse space.
LSA space: Projected to dense k-dimensional "semantic" space.



Synonyms cluster; polysemes may separate (or average, depending on corpus).

Latent Semantic Indexing

LSI applies LSA to information retrieval:

Indexing (offline):

1. Build term-document matrix $\mathbf{A}$ from corpus
2. Apply TF-IDF weighting
3. Compute truncated SVD: $\mathbf{A}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top$
4. Store $\mathbf{U}_k$, $\mathbf{\Sigma}_k$, and document vectors $\mathbf{V}_k$

Retrieval (online):

1. Convert query $\mathbf{q}$ to term vector (same weighting)
2. Project into latent space: $\hat{\mathbf{q}} = \mathbf{\Sigma}_k^{-1} \mathbf{U}_k^\top \mathbf{q}$
3. Rank documents by cosine similarity to $\hat{\mathbf{q}}$

Query Projection: Folding In

Problem: Query $\mathbf{q}$ is not in the original SVD.
Solution: "Fold in" the query as a pseudo-document.

If $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$, then document j's latent representation is:

$$\hat{\mathbf{d}}_j = \text{row } j \text{ of } \mathbf{V} = \mathbf{\Sigma}^{-1}\mathbf{U}^\top \mathbf{a}_j$$

For a new query vector $\mathbf{q}$:

$$\hat{\mathbf{q}} = \mathbf{\Sigma}_k^{-1}\mathbf{U}_k^\top \mathbf{q}$$

Intuition: We're finding the best coordinates for $\mathbf{q}$ in the existing latent space, weighted by dimension importance.

Similarity in Latent Space

Cosine similarity between query and document in latent space:

$$\text{sim}(\mathbf{q}, \mathbf{d}_j) = \frac{\hat{\mathbf{q}} \cdot \hat{\mathbf{d}}_j}{\|\hat{\mathbf{q}}\| \, \|\hat{\mathbf{d}}_j\|}$$

Why cosine?

- Length-normalized (document length doesn't matter)
- Measures angle between vectors
- Bounded: $-1 \leq \text{sim} \leq 1$

LSI advantage: Query "car insurance" can match document containing "automobile coverage" even with zero term overlap!

Term-Term Similarity

LSA provides term vectors: row i of $\mathbf{U}_k\mathbf{\Sigma}_k$ gives term i's representation.

Term similarity:

$$\text{sim}(t_i, t_j) = \cos(\mathbf{u}_i\mathbf{\Sigma}_k, \mathbf{u}_j\mathbf{\Sigma}_k)$$

Example similarities (Landauer & Dumais, 1997):

Word Pair	**LSA Similarity**
doctor – physician	0.79
car – automobile	0.82
gem – jewel	0.74
journey – voyage	0.71

LSA captures synonymy through distributional patterns.

Document-Document Similarity

Document vectors: rows of $\mathbf{V}_k\mathbf{\Sigma}_k$ (or equivalently, columns of $\mathbf{A}$ projected).

Applications:

- **Document clustering**: Group similar documents
- **Duplicate detection**: Find near-identical content
- **Recommendation**: "Documents like this one"
- **Corpus exploration**: Visualize document relationships

Visualization: Project to 2D using first two dimensions of $\mathbf{V}_k$.

(Though typically t-SNE or UMAP on full k dimensions works better.)

LSA for Essay Scoring

Landmark application (Landauer et al., 1998):

Method:

1. Train LSA on relevant domain corpus
2. Represent student essays as document vectors
3. Compare to "gold standard" essays via cosine similarity
4. Score correlates with human grades

Results:

- LSA-human agreement $\approx$ human-human agreement
- Captures content coverage, not just surface features
- Still used in educational assessment (e.g., ETS)

Limitations of LSA

Theoretical limitations:

- **Bag of words**: Ignores word order and syntax
- **Linear model**: Only captures linear relationships
- **Polysemy handling**: Single vector per word (averages senses)
- **No compositionality**: Can't represent phrases properly

Practical limitations:

- **Scalability**: SVD is $O(\min(nm^2, n^2m))$
- **Updates**: Adding documents requires re-computing SVD
- **Dimension selection**: Choosing k is somewhat arbitrary
- **Interpretability**: Latent dimensions are opaque

Choosing k

The rank hyperparameter:

- Too small: Loses important distinctions
- Too large: Overfits to noise / lexical specifics

Methods for selection:

1. **Held-out evaluation**: Tune on validation set (if task is known)
2. **Scree plot**: Look for "elbow" in singular value decay
3. **Explained variance**: Keep dimensions explaining $X\%$ of variance
4. **Rules of thumb**: $k \in [100, 300]$ often works for large corpora

Observation: Performance is often robust across a range of k values.

From LSA to Modern Embeddings

LSA's legacy:

- Established distributional semantics paradigm
- Showed that co-occurrence $\Rightarrow$ meaning
- Dense, low-dimensional representations

Evolution:



Key differences:

- Word2Vec: Prediction-based (not count-based)
- BERT: Contextual embeddings (different vector per usage)

Relationship to PMI and SVD

Surprising connection (Levy & Goldberg, 2014):

Word2Vec with skip-gram is implicitly factorizing a **shifted PMI matrix**:

$$\mathbf{M}_{ij} = \text{PMI}(w_i, c_j) - \log k$$

Implications:

- Neural embeddings $\approx$ matrix factorization + some tricks
- LSA on PPMI matrix performs comparably to Word2Vec
- The distributional hypothesis underlies all these methods

Takeaway: LSA isn't "outdated"—it's foundational. Understanding SVD helps you understand modern embeddings.

Key Takeaways

1. **LSA** uses SVD to discover latent semantic structure from co-occurrence.
2. **Truncated SVD** reduces dimensionality, capturing major semantic patterns while filtering noise.
3. **LSI** applies LSA to information retrieval, enabling semantic matching beyond keyword overlap.
4. **Limitations**: Bag-of-words, linear, single vector per word.
5. **Legacy**: LSA established the distributional paradigm that underlies Word2Vec, GloVe, and even contextual embeddings.

References

- Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). Indexing by latent semantic analysis. *JASIS*, 41(6), 391–407.
- Landauer, T. K., & Dumais, S. T. (1997). A solution to Plato's problem: The latent semantic analysis theory of acquisition, induction, and representation of knowledge. *Psychological Review*, 104(2), 211–240.
- Landauer, T. K., Foltz, P. W., & Laham, D. (1998). An introduction to latent semantic analysis. *Discourse Processes*, 25(2-3), 259–284.
- Levy, O., & Goldberg, Y. (2014). Neural word embedding as implicit matrix factorization. *NeurIPS*.
- Turney, P. D., & Pantel, P. (2010). From frequency to meaning: Vector space models of semantics. *JAIR*, 37, 141–188.

Questions?

Next: Neural Language Models